

CS 2300 - Homework 4

Problem 1: Company Database Queries

1. Retrieve the names of all employees in department 5 who work more than 10 hours per week on the 'ProductX' project.

- Select the ProductX project.

$$ProductX_Pno \leftarrow \pi_{Pnumber}(\sigma_{Pname='ProductX'}(PROJECT))$$

- Select the employees working more than 10 hours.

$$Over10_Hours_Essn \leftarrow \pi_{Essn, Pno}(\sigma_{Hours>10}(WORKS_ON))$$

- Find the employees who satisfy both conditions by joining the previous results.

$$ProductX_Workers \leftarrow ProductX_Pno \bowtie_{Pnumber=Pno} Over10_Hours_Essn$$

- Select employees from department 5.

$$Dept5_Emps \leftarrow \pi_{Ssn, Fname, Lname}(\sigma_{Dno=5}(EMPLOYEE))$$

- Join with the employees from department 5 to get their names.

$$Result \leftarrow Dept5_Emps \bowtie_{Ssn=Essn} ProductX_Workers$$

- Final Expression

$$\pi_{Fname, Lname}(Result)$$

2. List the names of all employees who have a dependent with the same first name as themselves.

- Join employees with their dependents.

$$Emp_Deps \leftarrow EMPLOYEE \bowtie_{Ssn=Essn} DEPENDENT$$

- Select the records where the names match.

$$Matching_Names \leftarrow \sigma_{Fname=Dependent_name}(Emp_Deps)$$

- Projecting Employee Names:

$$\pi_{Fname, Lname}(Matching_Names)$$

- Final Expression

$$\pi_{Fname, Lname}(\sigma_{Fname=Dependent_name}(EMPLOYEE \bowtie_{Ssn=Essn} DEPENDENT))$$

3. Find the names of all employees who are directly supervised by 'Franklin Wong'.

- Find the Ssn of 'Franklin Wong'.

$$Wong_Ssn \leftarrow \pi_{Ssn}(\sigma_{Fname='Franklin' \wedge Lname='Wong'}(EMPLOYEE))$$

- Select employees supervised by Franklin Wong using a join with renamed relations.

$$Result \leftarrow \rho_E(EMPLOYEE) \bowtie_{E.Super_ssn=S.Ssn} \sigma_{Fname='Franklin' \wedge Lname='Wong'}(\rho_S(EMPLOYEE))$$

- Final Expression

$$\pi_{E.Fname, E.Lname}(Result)$$

4. For each project, list the project name and the total hours per week (by all employees) spent on that project.

- Join PROJECT with WORKS_ON.

$$Proj_Work \leftarrow PROJECT \bowtie_{Pnumber=Pno} WORKS_ON$$

- Group by the project name and apply the SUM aggregate function to the Hours.
- Final Expression:

$$Pname \mathcal{F}_{SUM(Hours)}(Proj_Work)$$

5. Retrieve the names of all employees who work on every project. (Division is ALLOWED).

- Get all project numbers.

$$All_Projects \leftarrow \pi_{Pnumber}(PROJECT)$$

- Get all employee-project pairs from WORKS_ON.

$$Emp_Proj_Pairs \leftarrow \pi_{Essn, Pno}(WORKS_ON)$$

- Divide the pairs by the projects to find the **Essn** of employees who work on all of them.

$$Result_Ssn \leftarrow Emp_Proj_Pairs \div All_Projects$$

- Join with **EMPLOYEE** to get the names.
- Final Expression:

$$\pi_{Fname, Lname}(Result_Ssn \bowtie_{Essn=Ssn} EMPLOYEE)$$

Or, as a single expression:

$$\pi_{Fname, Lname}((\pi_{Essn, Pno}(WORKS_ON) \div \pi_{Pnumber}(PROJECT)) \bowtie_{Essn=Ssn} EMPLOYEE)$$

6. Retrieve the names of all employees who work on every project. (Division is NOT ALLOWED).

- Find all possible (*Ssn*, *Pnumber*) combinations.

$$All_Possible_Pairs \leftarrow \pi_{Ssn}(EMPLOYEE) \times \pi_{Pnumber}(PROJECT)$$

- Find the actual pairs that exist

$$Actual_Pairs \leftarrow \rho_{Ssn \leftarrow Essn, Pnumber \leftarrow Pno}(\pi_{Essn, Pno}(WORKS_ON))$$

- Find the pairs that are missing from the **WORKS_ON** table

$$Missing_Pairs \leftarrow All_Possible_Pairs - Actual_Pairs$$

- Find the **Ssn** of all employees who are missing at least one project.

$$Incomplete_Emps \leftarrow \pi_{Ssn}(Missing_Pairs)$$

- Find all employees and subtract the incomplete ones.

$$Complete_Emps_Ssn \leftarrow \pi_{Ssn}(EMPLOYEE) - Incomplete_Emps$$

- Join with **EMPLOYEE** to get their names.
- Final Expression:

$$\pi_{Fname, Lname}(Complete_Emps_Ssn \bowtie EMPLOYEE)$$

7. Retrieve the names of all employees who do not work on any project.

- Get the **Ssn** of all employees.

$$All_Emps \leftarrow \pi_{Ssn}(EMPLOYEE)$$

- Get the **Essn** of all employees who work on at least one project.

$$Working_Emps \leftarrow \pi_{Essn}(WORKS_ON)$$

- Find the employees who are in **All_Emps** but not in **Working_Emps**

$$Non_Working_Ssn \leftarrow All_Emps - \rho_{Ssn \leftarrow Essn}(Working_Emps)$$

- Join back to the **EMPLOYEE** table to get their names.
- Final Expression:

$$\pi_{Fname, Lname}(Non_Working_Ssn \bowtie EMPLOYEE)$$

8. For each department, retrieve the department name and the average salary of all employees working in that department.

- Join **DEPARTMENT** and **EMPLOYEE**.

$$Dept_Emps \leftarrow DEPARTMENT \bowtie_{Dnumber=Dno} EMPLOYEE$$

- Group by **Dname** and apply the **AVG** function to **Salary**.
- Final Expression:

$$Dnumber, Dname \mathcal{F}_{AVG(Salary)}(Dept_Emps)$$

9. Retrieve the average salary of all female employees.

- Select all female employees.

$$Female_Emps \leftarrow \sigma_{Sex='F'}(EMPLOYEE)$$

- Apply the **AVG** aggregate function to the result.
- Final Expression:

$$\mathcal{F}_{AVG(Salary)}(Female_Emps)$$

10. Retrieve all employees by name and SSN who have cumulatively worked more than 100 hours across all projects. For each qualifying employee, also include the total hours worked and the name and manager SSN of their respective departments. Additionally, the result should account for departments that lack employees who meet this threshold.

- Calculate the total hours for each employee from WORKS_ON.

$$Emp_Total_Hours \leftarrow_{Essn} \mathcal{F}_{SUM}(Hours) \text{ as } TotalHours (WORKS_ON)$$

- Select only those employees who have worked more than 100 hours.

$$Qualifying_Emps \leftarrow \sigma_{TotalHours > 100}(Emp_Total_Hours)$$

- Join this result with the EMPLOYEE table to get employee details.

$$Qualifying_Emp_Info \leftarrow EMPLOYEE \bowtie_{Ssn=Essn} Qualifying_Emps$$

- Perform a right outer join with the DEPARTMENT table.

$$Final_Join \leftarrow Qualifying_Emp_Info \bowtie_{Dno=Dnumber} DEPARTMENT$$

- Project the required attributes from the result of the outer join.
- Final Expression:

$$\pi_{Fname, Lname, Ssn, TotalHours, Dname, Mgr_ssn}(Final_Join)$$

Problem 2: Airline Database Queries

a. For each flight, list the flight number, the departure airport for the first leg of the flight, and the arrival airport for the last leg of the flight.

- Find the departure airport for the first leg of each flight.

$$First_Legs \leftarrow \pi_{Flight_number, Departure_airport_code}(\sigma_{Leg_number=1}(FLIGHT_LEG))$$

- Find the maximum leg number for each flight.

$$Last_Leg_Numbers \leftarrow Flight_number \mathcal{F}_{MAX(Leg_number)} \text{ as } Max_Leg(FLIGHT_LEG)$$

- Find the arrival airport for the last leg of each flight by joining the max leg numbers back to the FLIGHT_LEG table.

Last_Legs

$$\begin{aligned} &\leftarrow \pi_{FL.Flight_number, Arrival_airport_code}(\rho_{FL}(FLIGHT_LEG) \\ &\bowtie_{FL.Flight_number=LLN.Flight_number \wedge FL.Leg_number=LLN.Max_Leg} \rho_{LLN}(Last_Leg_Numbers)) \end{aligned}$$

- Join the first leg and last leg information together on the flight number.
- Final Expression:

$$Result \leftarrow First_Legs \bowtie Last_Legs$$

b. List the flight numbers and weekdays of all flights or flight legs that depart from Houston Intercontinental Airport (airport code 'iah') and arrive in Los Angeles International Airport (airport code 'lax').

- Select all flight legs from IAH to LAX from the FLIGHT_LEG table.

$$IAH_to_LAX_Legs \leftarrow \sigma_{Departure_airport_code='iah' \wedge Arrival_airport_code='lax'}(FLIGHT_LEG)$$

- Join the result with the FLIGHT table on Flight_number to get the weekdays.

$$Result_Join \leftarrow FLIGHT \bowtie_{Flight_number} IAH_to_LAX_Legs$$

- Project the required attributes.
- Final Expression:

$$\pi_{Flight_number, Weekdays}(Result_Join)$$

c. List the flight number, departure airport code, scheduled departure time, arrival airport code, scheduled arrival time, and weekdays of all flights or flight legs that depart from some airport in the city of Houston and arrive at some airport in the city of Los Angeles.

- Find all airport codes in the city of Houston.

$$Houston_Airports \leftarrow \pi_{Airport_code}(\sigma_{City='Houston'}(AIRPORT))$$

- Find all airport codes in the city of Los Angeles.

$$LA_Airports \leftarrow \pi_{Airport_code}(\sigma_{City='LosAngeles'}(AIRPORT))$$

- Find all flight legs that depart from any Houston airport.

$$Legs_From_Houston \leftarrow FLIGHT_LEG \bowtie_{Departure_airport_code=Airport_code} Houston_Airports$$

- From that result, find the legs that arrive at any Los Angeles airport.

$$Houston_to_LA_Legs \leftarrow Legs_From_Houston \bowtie_{Arrival_airport_code=Airport_code} LA_Airports$$

- Join with the FLIGHT table to get the weekdays for these flights.

$$Result_Join \leftarrow FLIGHT \bowtie_{Flight_number} Houston_to_LA_Legs$$

- Project the required attributes.
- Final Expression:

$$\pi_{Flight_number, Departure_airport_code, Scheduled_departure_time, Arrival_airport_code, Scheduled_arrival_time, Weekdays}(Result_Join)$$

d. List all fare information for flight number 'c0197'.

- Select all records from the FARE table where the Flight_number is 'c0197'.
- Final Expression:

$$\sigma_{Flight_number='c0197'}(FARE)$$

e. Retrieve the number of available seats for flight number 'c0197' on '2009-10-09'.

- Select all leg instances for the given flight and date.

$$Flight_Instances \leftarrow \sigma_{Flight_number='c0197' \wedge Date='2009-10-09'}(LEG_INSTANCE)$$

- Project the Number_of_available_seats attribute. Also project Leg_number to distinguish between the legs.
- Final Expression:

$$\pi_{Leg_number, Number_of_available_seats}(Flight_Instances)$$

Problem 3: Actor's Latest Movie Query

Write a relational algebra expression that will produce a resulting relation of each actor's latest film and their age when they starred in it—i.e. a relation of the following scheme: `ActorLatestMovie(actor_name:str, age_during_movie:int, latest_movie_title:str, movie_year:int)`. Assume there exists some actors who haven't starred in any movies. They ought to appear in the relation.

- Create a unified view of actors and the movies they starred in.

$$\begin{aligned} R_1 &\leftarrow \rho_{A(aid, name, birthyear)}(Actor) \\ &\quad \bowtie_{A.aid=S.aid} \rho_S(STARS_IN) \\ &\quad \bowtie_{S.mid=M.mid} \rho_M(mid, title, year)(Movie) \end{aligned}$$

- Find the latest movie year for each actor.

$$R_2 \leftarrow \text{aid} \mathcal{F}_{\text{MAX}(year)} \text{ as } latest_movie_year(R_1)$$

- Combine all actors with their latest movie year.

$$R_3 \leftarrow \rho_{A(aid, name, birthyear)}(Actor) \bowtie_{A.aid=R_2.aid} R_2$$

- Retrieve the title of the latest movie.

$$R_4 \leftarrow R_3 \bowtie_{R_3.aid=R_1.aid \wedge R_3.latest_movie_year=R_1.year} R_1$$

- Calculate the age and project to the desired attributes.

$$R_5 \leftarrow \pi_{name, (latest_movie_year - birthyear), title, latest_movie_year}(R_4)$$

- Rename the final relation and its attributes to match the target schema.

$$Answer \leftarrow \rho_{ActorLatestMovie(actor_name, age_during_movie, latest_movie_title, movie_year)}(R_5)$$